

Лабораторная работа №8. Создание приложения с прокручиваемым списком

Цель работы: Научиться создавать прокручиваемые списки в Android-приложениях с использованием Jetpack Compose. В результате лабораторной работы будет создано приложение "Anime Catalog" (Каталог аниме), которое отображает список популярных аниме с изображениями и описаниями.

Теоретическая часть

LazyColumn и списки в Jetpack Compose

LazyColumn — это компонент Jetpack Compose для создания вертикальных прокручиваемых списков. Основные особенности:

- **Ленивая загрузка (Lazy loading)** — элементы создаются только когда они появляются на экране, что экономит память
- **Эффективная прокрутка** — отрисовываются только видимые элементы
- **Автоматическая переработка (recycling)** — элементы, покинувшие экран, используются повторно

Основные компоненты приложения

В данной работе мы будем использовать:

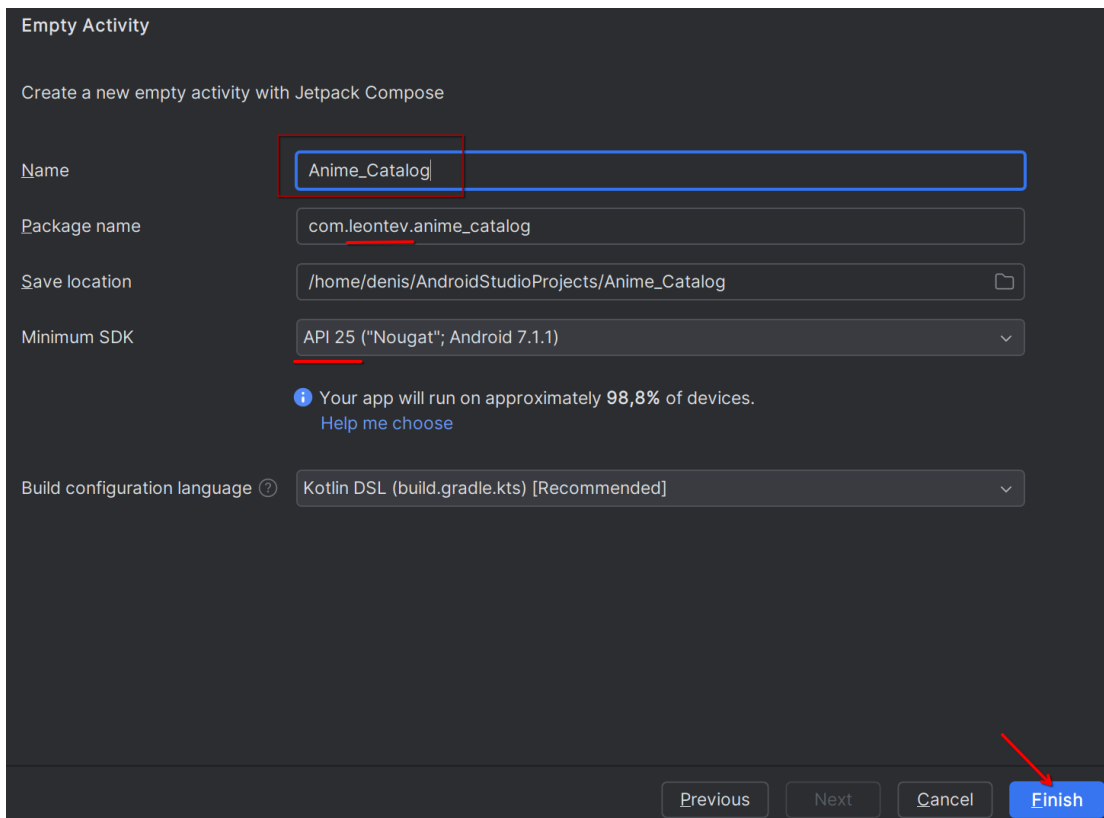
- **LazyColumn** — прокручиваемый список
- **Card** — карточка для отображения элемента списка
- **Image** — отображение изображения
- **Text** — отображение текстовой информации
- **Data class** — модель данных для элемента списка
- **Datasource** — источник данных

Практическая часть

Шаг 1. Создание проекта

1. Откройте Android Studio
2. Выберите **File** → **New** → **New Project**
3. В окне выбора шаблона выберите **Empty Activity** → **Next**
4. Укажите параметры:
 - **Name:** Anime_Catalog
 - **Package name:** com.example.animecatalog
 - **Minimum SDK:** API 25

5. Нажмите **Finish**



Empty Activity

Create a new empty activity with Jetpack Compose

Name: Anime_Catalog

Package name: com.leontev.anime_catalog

Save location: /home/denis/AndroidStudioProjects/Anime_Catalog

Minimum SDK: API 25 ("Nougat"; Android 7.1.1)

Your app will run on approximately 98,8% of devices.
[Help me choose](#)

Build configuration language: Kotlin DSL (build.gradle.kts) [Recommended]

Previous Next Cancel **Finish**

Дождитесь полной загрузки проекта.

Шаг 2. Подготовка ресурсов

2.1. Добавление изображений

Способ 1: Перетаскивание (рекомендуется)

1. Скачайте 10 изображений аниме на компьютер

Рекомендации по изображениям:

- Формат: **JPG** или **PNG**
- Минимальная ширина: **800px** (для качественного отображения)
- Максимальный размер файла: **500KB** (чтобы не раздувать APK)
- Пропорции: желательно **16:9** или близкие к этому

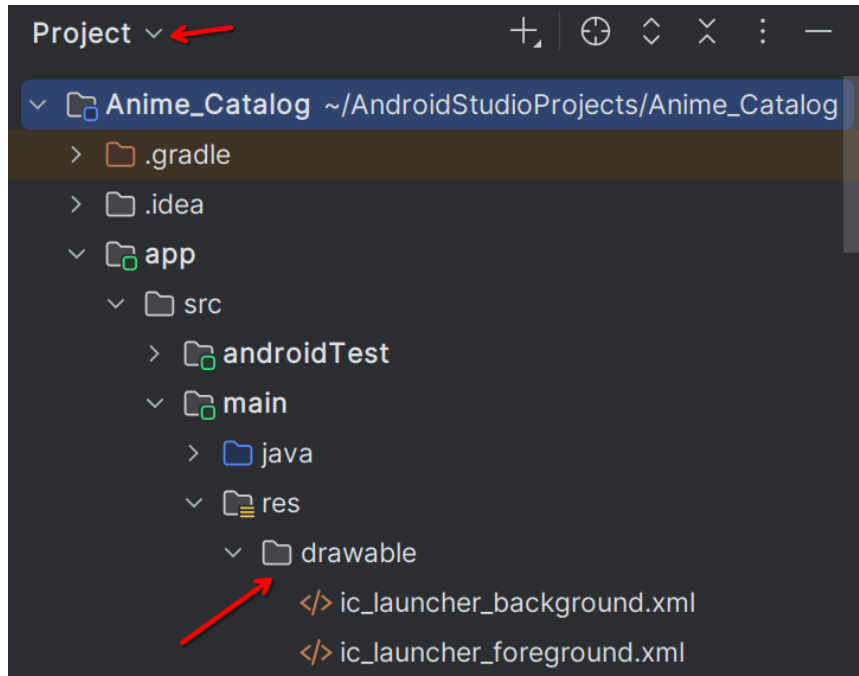
Список файлов для добавления (можете использовать свои):

- anime1.jpg или anime1.png — Death Note
- anime2.jpg или anime2.png — Attack on Titan
- anime3.jpg или anime3.png — Naruto
- anime4.jpg или anime4.png — One Piece
- anime5.jpg или anime5.png — Demon Slayer
- anime6.jpg или anime6.png — My Hero Academia
- anime7.jpg или anime7.png — Sword Art Online

- anime8.jpg или anime8.png — Fullmetal Alchemist
- anime9.jpg или anime9.png — Tokyo Ghoul
- anime10.jpg или anime10.png — Steins;Gate

2. В Android Studio переключитесь в режим просмотра **Project** (слева вверху)

3. Найдите папку **app/src/main/res/drawable**



4. **Перетащите** все 10 изображений в эту папку

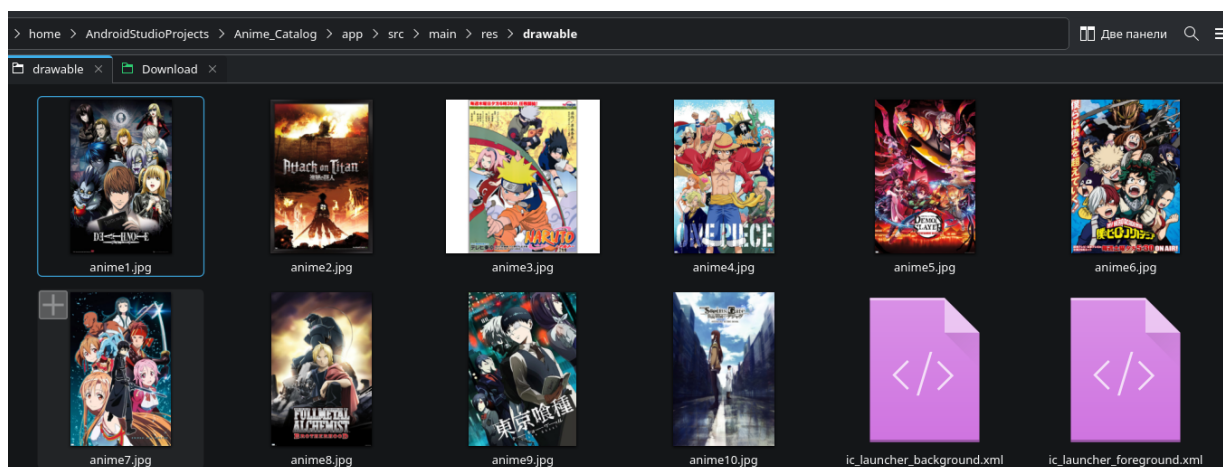
5. В диалоговом окне выберите **OK**

Способ 2: Через контекстное меню

1. Кликните правой кнопкой на папке **res/drawable**

2. Выберите **Open In - Explorer**

3. Скопируйте изображения в открывшуюся папку



4. Вернитесь в Android Studio

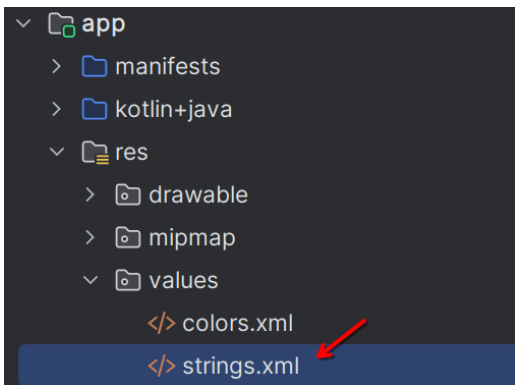
5. Папка **drawable** автоматически обновится

Почему НЕ использовать Resource Manager?

Resource Manager предназначен для векторных изображений и иконок. Для обычных фотографий/постеров проще использовать прямое добавление в папку drawable.

2.2. Настройка strings.xml

Откройте `res/values/strings.xml`



Добавьте следующие строки:

```
<resources>
    <string name="app_name">Anime_Catalog</string>
    <!-- Названия аниме -->
    <string name="anime1">Death Note</string>
    <string name="anime2">Attack on Titan (Shingeki no Kyojin)</string>
</resources>
```

Добавьте весь **остальной список** аниме, чтобы у вас их стало **10**:

- anime1 — Death Note;
- anime2 — Attack on Titan (Shingeki no Kyojin);
- anime3 — Naruto;
- anime4 — One Piece;
- anime5 — Demon Slayer (Kimetsu no Yaiba);
- anime6 — My Hero Academia (Boku no Hero Academia);
- anime7 — Sword Art Online;
- anime8 — Fullmetal Alchemist: Brotherhood;
- anime9 — Tokyo Ghoul;
- anime10 — Steins;Gate;

Далее добавим описание для аниме:

```

<resources>
    <string name="app_name">Anime_Catalog</string>
    <!-- Названия аниме -->
    ...
    <!-- Описания аниме -->
    <string name="anime1_description">Гениальный студент находит таинственную
+   тетрадь смерти и решает создать идеальный мир.</string>
    <string name="anime2_description">Человечество борется за выживание против
    гигантских титанов, пожирающих людей.</string>
</resources>

```

Вам нужно добавить 10 описаний, для всех аниме. Примерный текст для вставки:

Гениальный студент находит таинственную тетрадь смерти и решает создать идеальный мир.

Человечество борется за выживание против гигантских титанов, пожирающих людей.

История молодого ниндзя Наруто, мечтающего стать Хокаге своей деревни.

Приключения Луффи и его пиратской команды в поисках легендарного сокровища.

Танджиро сражается с демонами, чтобы спасти свою сестру и отомстить за семью.

В мире супергероев подросток без способностей мечтает стать величайшим героем.

Игроки оказываются в ловушке виртуальной реальности, где смерть в игре означает смерть в реальности.

Два брата-алхимика ищут философский камень, чтобы вернуть утраченные тела.

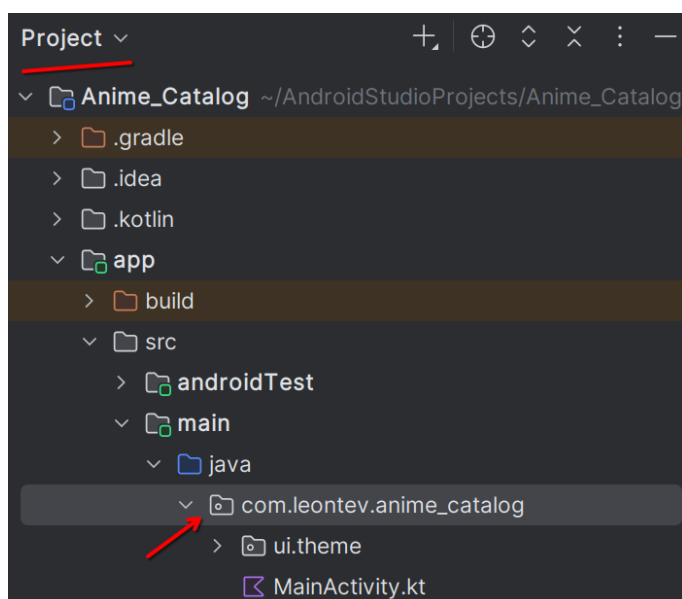
Студент Канеки превращается в полу-гуля после трагического инцидента.

Ученый случайно изобретает машину времени и сталкивается с последствиями.

Шаг 3. Создание модели данных (data class)

3.1. Создание пакета model

1. В Android Studio откройте структуру проекта
2. Найдите **app/src/main/java/com.fio.anime_catalog**

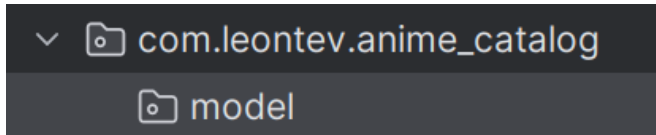


3. Кликните правой кнопкой на вашем пакете **animecatalog**

4. Выберите **New** → **Package**
5. Допишите имя пакета: **model**

```
com.leontev.anime_catalog.model
```

6. Нажмите **OK**



3.2. Создание класса **Anime**

1. Кликните правой кнопкой на созданном пакете **model**
2. Выберите **New** → **Kotlin Class/File**
3. Выберите **Class**
4. Введите имя: **Anime**
5. Нажмите **Enter**

Откроется файл **Anime.kt**. Обновите класс:

```
data class Anime (  
    @StringRes val titleResourceId: Int,  
    @StringRes val descriptionResourceId: Int,  
    @DrawableRes val imageResourceId: Int  
)
```

При необходимости добавьте импорты:

```
import androidx.annotation.DrawableRes  
import androidx.annotation.StringRes
```

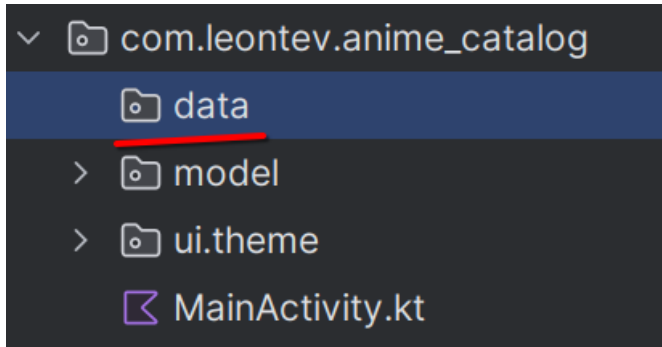
Объяснение:

- **data class** автоматически создает методы **equals()**, **hashCode()**, **toString()**, **copy()**
- **@StringRes** — аннотация, указывающая, что это ID строкового ресурса из **strings.xml**
- **@DrawableRes** — аннотация для ID ресурса изображения из папки **drawable**
- **titleResourceId** — ID названия аниме
- **descriptionResourceId** — ID описания аниме
- **imageResourceId** — ID изображения аниме

Шаг 4. Создание источника данных (Datasource)

4.1. Создание пакета data

1. Кликните правой кнопкой на папке **app/src/main/java/com.fio.anime_catalog**
2. Выберите **New → Package**
3. Допишите имя пакета: **data**
4. Нажмите **OK**



4.2. Создание класса Datasource

1. Кликните правой кнопкой на созданном пакете data
2. Выберите **New → Kotlin Class/File**
3. Выберите **Class**
4. Введите имя: **Datasource**
5. Нажмите **Enter**

Откроется файл **Datasource.kt**.

Добавьте необходимые импорты:

```
import com.leontev.anime_catalog.R
import com.leontev.anime_catalog.model.Anime
```

Обновите класс:

```
class Datasource {
    fun loadAnimeList(): List<Anime> {
        return listOf(
            Anime(
                titleResourceId = R.string.anime1,
                descriptionResourceId = R.string.anime1_description,
                imageResourceId = R.drawable.anime1
            ),
        )
    }
}
```

Также обратите внимание, что мы может не прописывать свойства. Так добавим второе аниме:

```
return listOf(  
    Anime(  
        titleResourceId = R.string.anime1,  
        descriptionResourceId = R.string.anime1_description,  
        imageResourceId = R.drawable.anime1  
    ),  
    Anime( titleResourceId = R.string.anime2,  
        descriptionResourceId = R.string.anime2_description,  
        imageResourceId = R.drawable.anime2)  
)
```

IDE сама поймёт какие свойства используется.

Для отключения подсказок, вы можете нажать **Alt + Enter**, и выбрать пункт:

Disable 'Parameters' Inlay Hints

Ваша задача добавить список из 10 аниме!

Объяснение:

- **class Datasource** — обычный класс, который служит источником данных
- **loadAnimeList()** возвращает список из **10 объектов Anime**
- **listOf()** создает неизменяемый список
- Каждый объект **Anime** содержит три параметра:
 - **ID названия** из **strings.xml**
 - **ID описания** из **strings.xml**
 - **ID изображения** из папки **drawable**
- **R.string.anime1** — это ссылка на строковый ресурс
- **R.drawable.anime1** — это ссылка на ресурс изображения

Шаг 5. Создание интерфейса

5.1. Очистка MainActivity.kt

Откройте **MainActivity.kt** и удалите лишние функции, оставив базовую структуру:


```

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            Anime_CatalogTheme {

            }
        }
    }
}

```

Нажмите **Ctrl+Alt+O** для оптимизации импортов.

```

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import com.leontev.anime_catalog.ui.theme.Anime_CatalogTheme

```

5.2. Создание карточки аниме

Создайте **Composable-функцию AnimeCard()** для отображения отдельной карточки:

```

@Composable
fun AnimeCard(anime: Anime, modifier: Modifier = Modifier) {

}

```

Далее при наличии ошибок добавляйте самостоятельно все необходимые импорты:

```

import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import com.leontev.anime_catalog.model.Anime

```

Внутри функции **AnimeCard()** добавим встроенную функцию для отображения наших карточек:

```

Card(modifier = modifier) {

}

```

Теперь внутри создадим **вертикальное** размещение элементов:

```
Card(modifier = modifier) {  
    +Column { }  
}
```

Нашу карточку мы хотим сделать следующим образом:

- 1 — изображение;
- 2 — Текст с названием аниме;
- 3 — описание аниме.



Добавим соответствующие **Composable-функции**, соглашаясь на импорт:

```
Column {  
    Image()  
    Text()  
    Text()  
}
```

Добавим в наше изображение **Image** следующие параметры:

```
Image(
    painter = painterResource(anime.imageResourceId),
    contentDescription = stringResource(anime.titleResourceId),
    modifier = Modifier
        .fillMaxWidth()
        .height(450.dp),
    contentScale = ContentScale.Crop
)
```

- **contentDescription** — берём из нашего xml файла;
- **fillMaxWidth()** — изображение занимает всю ширину;
- **height(450.dp)** — фиксированная высота;
- **contentScale = ContentScale.Crop** — обрезка изображения для заполнения области

Первый Text отображает название аниме:

```
Text(
    text = stringResource(anime.titleResourceId),
    modifier = Modifier.padding(16.dp),
    style = MaterialTheme.typography.headlineSmall
)
```

Второй Text отображает описание:

```
Text(
    text = stringResource(anime.descriptionResourceId),
    modifier = Modifier.padding(start = 16.dp, end = 16.dp, bottom = 16.dp),
    style = MaterialTheme.typography.bodyMedium
)
```

5.3. Создание списка (LazyColumn)

Добавьте функцию для отображения списка:

```
@Composable
fun AnimeList(animeList: List<Anime>, modifier: Modifier = Modifier) {
    LazyColumn(modifier = modifier) {
        items(animeList) { anime ->
            AnimeCard(
                anime = anime,
                modifier = Modifier.padding(8.dp)
            )
        }
    }
}
```

Объяснение:

- **LazyColumn** создаёт прокручиваемый список
- **items(animeList)** перебирает каждый элемент списка
- Для каждого элемента создаётся **AnimeCard**
- **modifier = Modifier.padding(8.dp)** добавляет отступы между карточками

5.4. Создание главной Composable-функции

Добавьте функцию **AnimeCatalogApp()**:

```
@Composable
fun AnimeCatalogApp() {
    AnimeList(
        animeList = Datasource().loadAnimeList()
    )
}
```

Объяснение:

- Функция загружает данные из **Datasource**
- Передаёт список в компонент **AnimeList**

5.5. Запуск функции в MainActivity

Запустим нашу главную функцию:

```
Anime_CatalogTheme {
    Surface(
        modifier = Modifier.fillMaxSize(),
        color = MaterialTheme.colorScheme.background
    ) {
        AnimeCatalogApp()
    }
}
```

Шаг 6. Добавление Preview

Добавьте функцию для предварительного просмотра карточки:

```

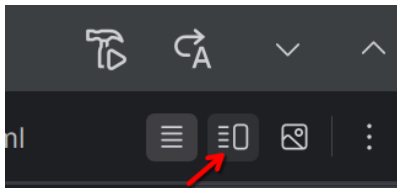
@Preview(showBackground = true)
@Composable
private fun AnimeCardPreview() {
    Anime_CatalogTheme {
        AnimeCard(
            Anime(
                R.string.anime1,
                R.string.anime1_description,
                R.drawable.anime1
            )
        )
    }
}
}

```

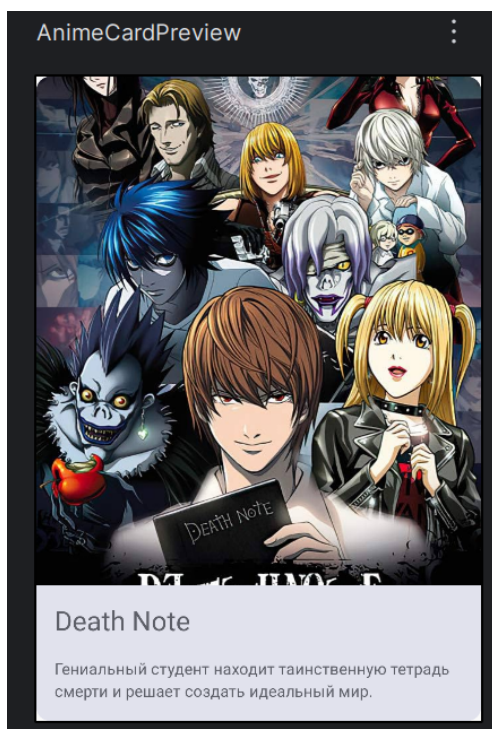
Объяснение:

- **@Preview** позволяет просмотреть компонент без запуска приложения
- **showBackground = true** отображает фон
- Создаётся тестовый объект Anime для превью

Запустите и проверьте отображение в предварительное просмотре:

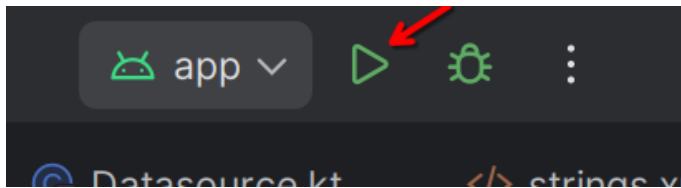


Итог:

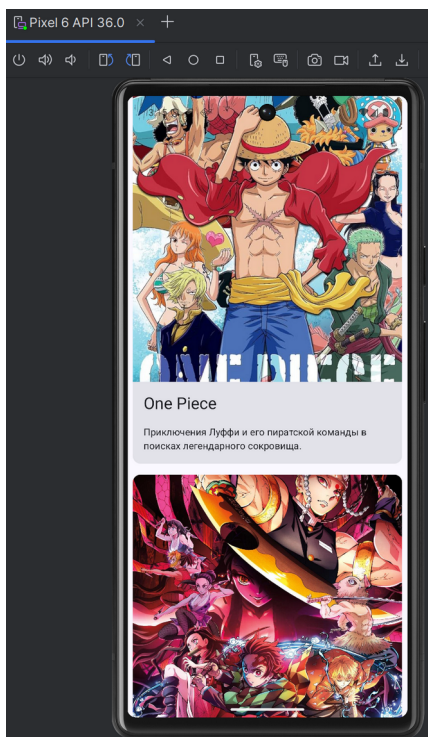


Шаг 7. Запуск приложения

1. Убедитесь, что все файлы сохранены
2. Выберите эмулятор или подключённое устройство
3. Нажмите **Run**:



Приложение должно отобразить прокручиваемый список из 10 аниме с изображениями, названиями и описаниями.



Шаг 8. Создание собственной иконки приложения в Krita

Теперь создадим **уникальную иконку приложения** для нашего аниме-каталога. Мы сделаем её **в аниме-стилистике** и подключим в Android Studio как **Adaptive Icon**.

8.1. Что такое Adaptive Icon?

Adaptive Icon — это иконка приложения, состоящая из **двух отдельных слоёв**:

- **Foreground (передний план)** — основной символ иконки
- **Background (задний план)** — фон

Android сам накладывает маску (круг, квадрат, скруглённый квадрат) в зависимости от устройства.

Почему используются два слоя?

- У разных производителей разные формы иконок
- Система может анимировать слои отдельно
- Создаётся эффект глубины и параллакса

Поэтому **фон и символ всегда делаются отдельно**

8.2. Подготовка Krita

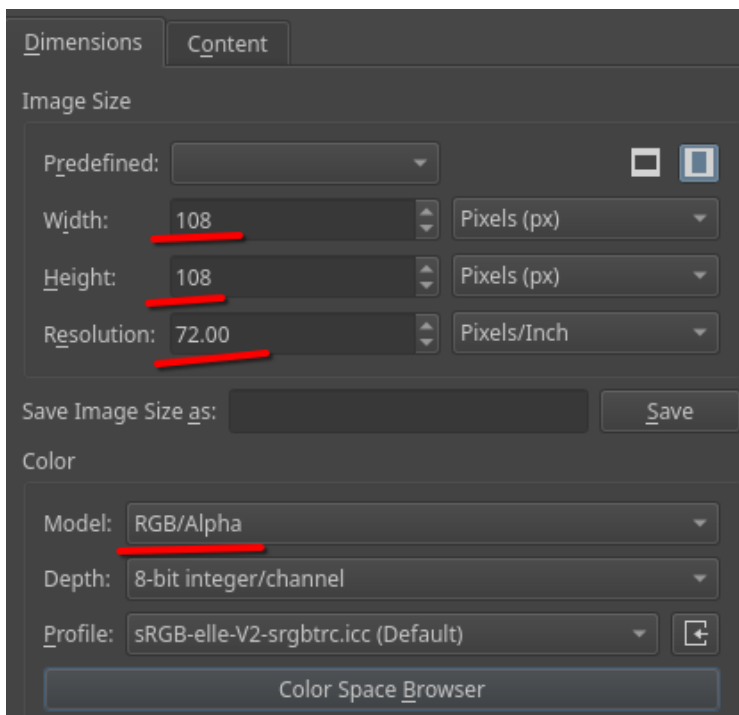
В данной лабораторной работе в качестве примера используется бесплатный редактор **Krita**, но вы можете использовать другие аналоги — Canva, Icon Kitchen, Figma или даже Paint.

1. Проверьте что на вашем ПК установлена **Krita**
2. Откройте редактор **Krita**
3. Выберите **File → New**

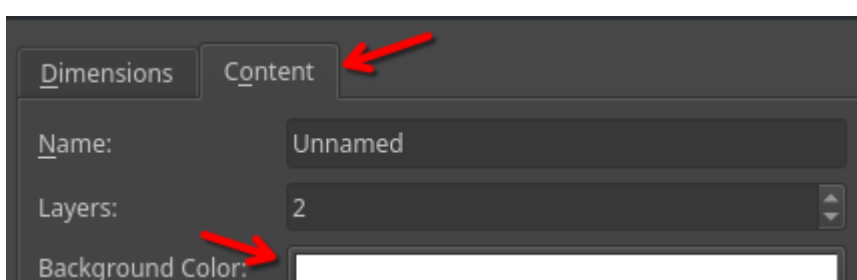
8.3. Создание Background (фон)

Шаг 1. Создание документа

1. В окне создания документа укажите:



- **Width:** 108 px
- **Height:** 108 px
- **Resolution:** 72 ppi
- **Color model:** RGBA
- **Background:** White



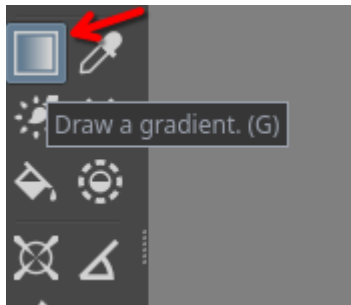
2. Нажмите **Create**

108×108 px — базовый размер для Adaptive Icon (mdpi)

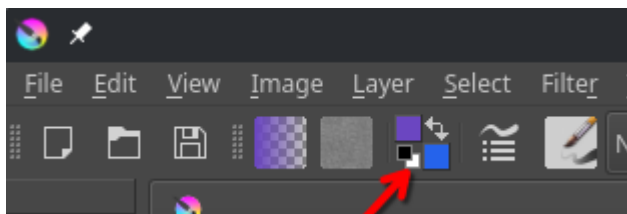
Шаг 2. Рисование фона

Вариант 1: Градиентный аниме-фон

1. Выберите инструмент **Gradient Tool**



2. Поменяйте цвета:



3. Пример цветов:

- Фиолетовый: #6B46C1
- Синий: #2563EB

4. Протяните градиент сверху вниз или по диагонали:



При желании:

- добавьте точки
- полосы
- лёгкий шум (Filter → Noise)

Вариант 2: Японский стиль

1. Залейте фон цветом:

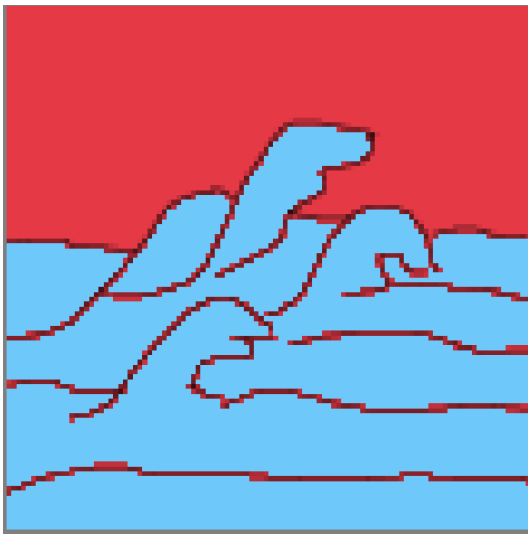
- Красный: #E63946



2. Добавьте узоры:

- волны
- сакуру
- геометрические элементы

Фон может заполнять весь холст, безопасная зона для него не требуется.



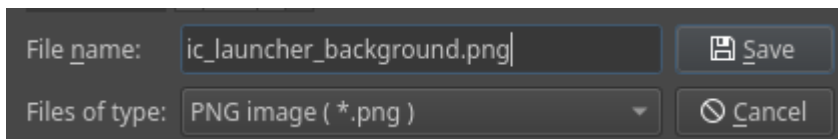
Вариант 3: Свой стиль

Нарисуйте свой вариант.

Шаг 3. Экспорт фона

1. **File** → **Export**
2. Формат: **PNG**
3. Имя файла:

ic_launcher_background.png



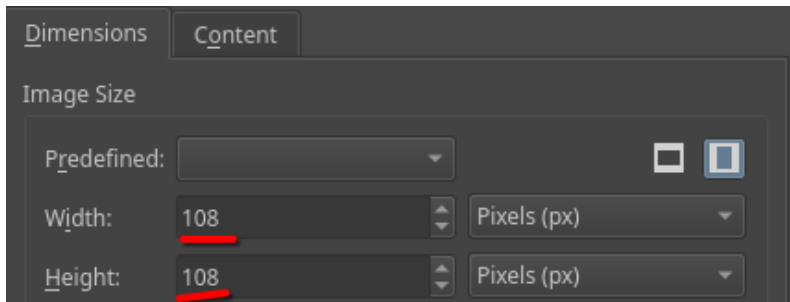
Оставляем опции по умолчанию и нажимаем **ОК**.

8.4. Создание Foreground (передний план)

Шаг 1. Новый документ

Создайте **новый файл**:

- **Width:** 108 px
- **Height:** 108 px



Шаг 2. Safe Zone (очень важно)

Основной элемент должен находиться в безопасной зоне

- Safe Zone — круг диаметром ~66 px
- Всё важное должно быть внутри него
- Иначе часть иконки может быть обрезана

В Krita:

- включите линейки (View → Show Rulers)
- ориентируйтесь на центр холста

Шаг 3. Создание иконки

Вариант 1: Текстовая иконка

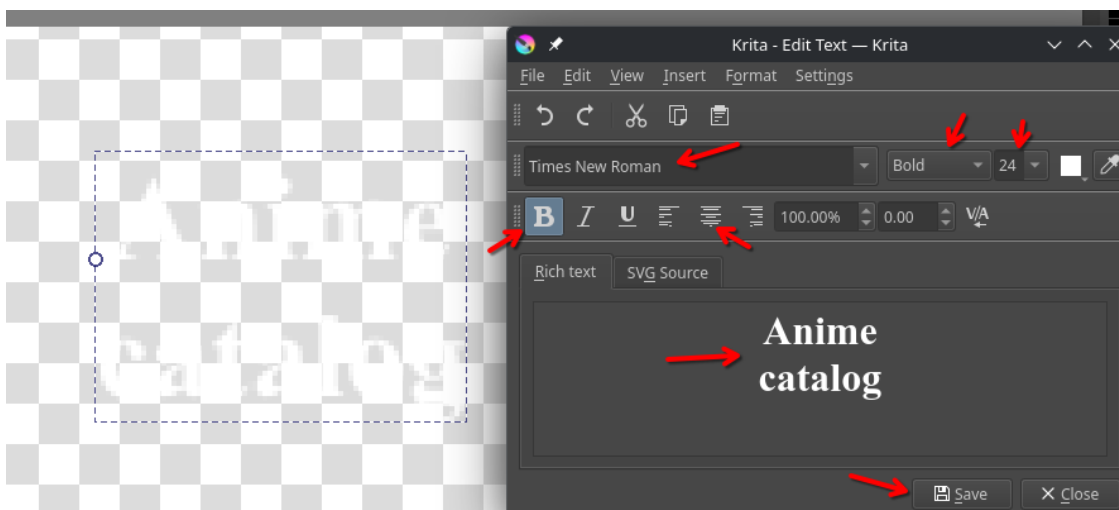
1. Выберите **Text Tool**



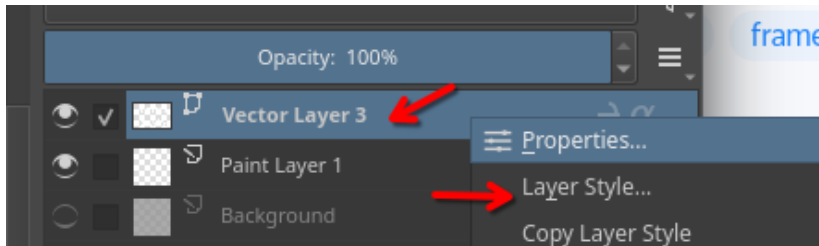
2. Выделите область, где будете писать.

3. В открывшемся редакторе введите букву Anime catalog

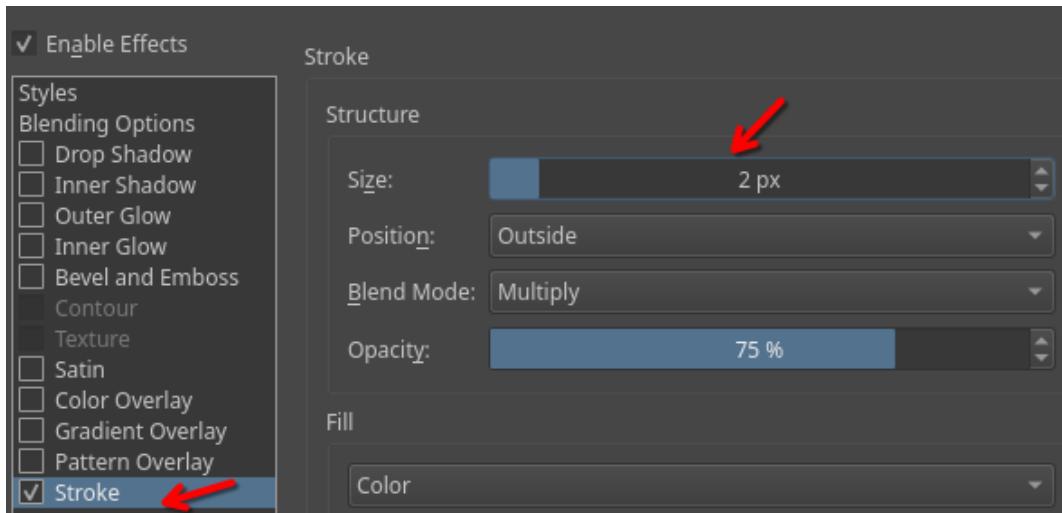
4. Можете использовать мои настройки, или поэкспериментировать со своими:



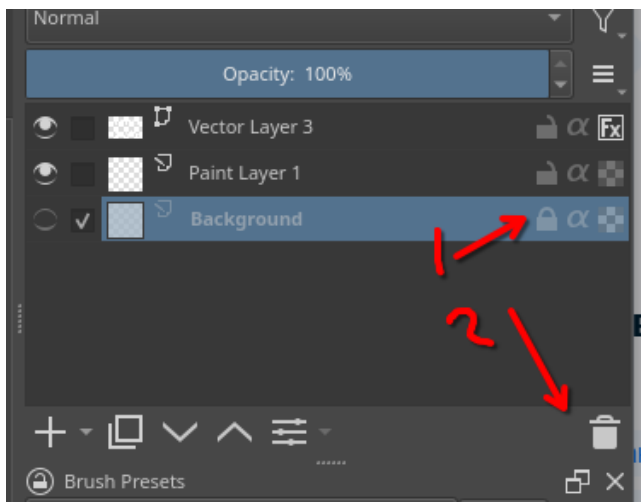
5. Можно добавить чёрную обводку. Для этого щёлкните по слою и выберите **Layer Style**:



6. Поставьте галочку у **Stroke** и **size** поменяйте на **2px**. Нажмите **ОК**:



7. Далее нам нужно снять **замок** и начального слоя и **удалить** его:



Вариант 2: Символ аниме

- Простой аниме-глаз
- Минималистичное лицо
- Используйте:
 - круги
 - линии
 - минимум деталей

Чем проще — тем лучше читается иконка

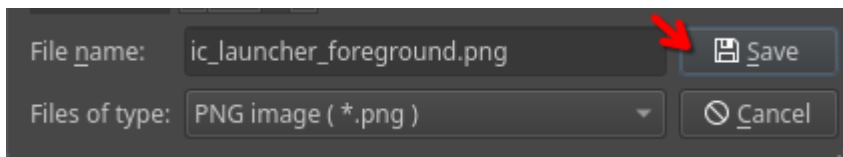
Вариант 3: Хлопушка (Clapperboard)

- Ассоциация с сериалами и аниме
- Простой и узнаваемый символ

Шаг 4. Экспорт переднего плана

1. **File** → **Export**
2. Формат: **PNG**
3. Имя файла:

ic_launcher_foreground.png



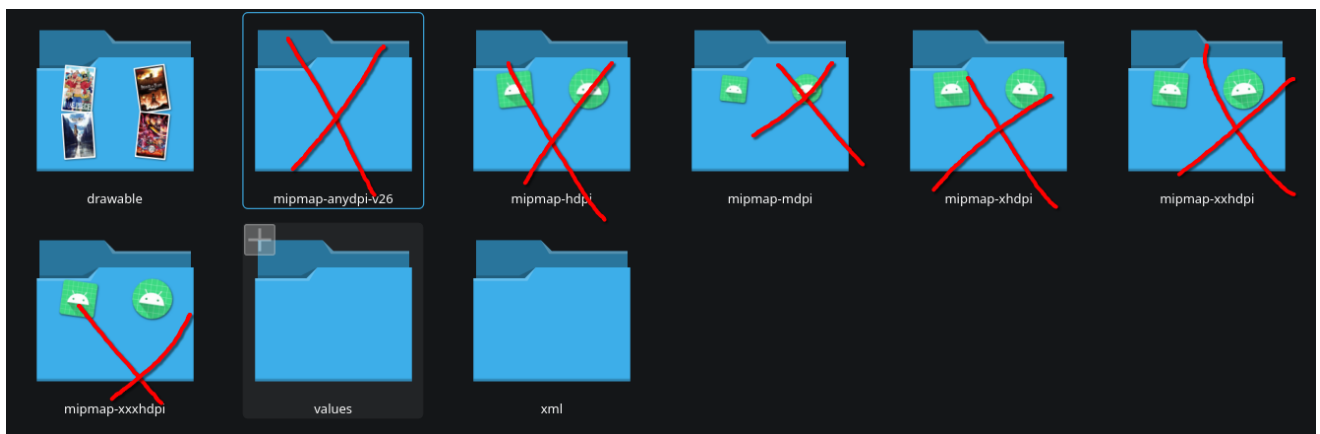
Оставляем опции по умолчанию и нажимаем **OK**.

8.5. Создание иконки в Android Studio

Удаление старых иконок

В папке **res** удалите **папки**:

- mipmap-anydpi-v26
- mipmap-hdpi
- mipmap-mdpi
- mipmap-xhdpi
- mipmap-xxhdpi
- mipmap-xxxhdpi



Создание новой иконки

1. Правый клик на **res**
2. **New** → **Image Asset**

Настройки:

Icon Type: Launcher Icons (Adaptive and Legacy)

Name: ic_launcher

Foreground Layer

- Asset Type: Image
- Path: ic_launcher_foreground.png
- Resize: 90–100%

Background Layer

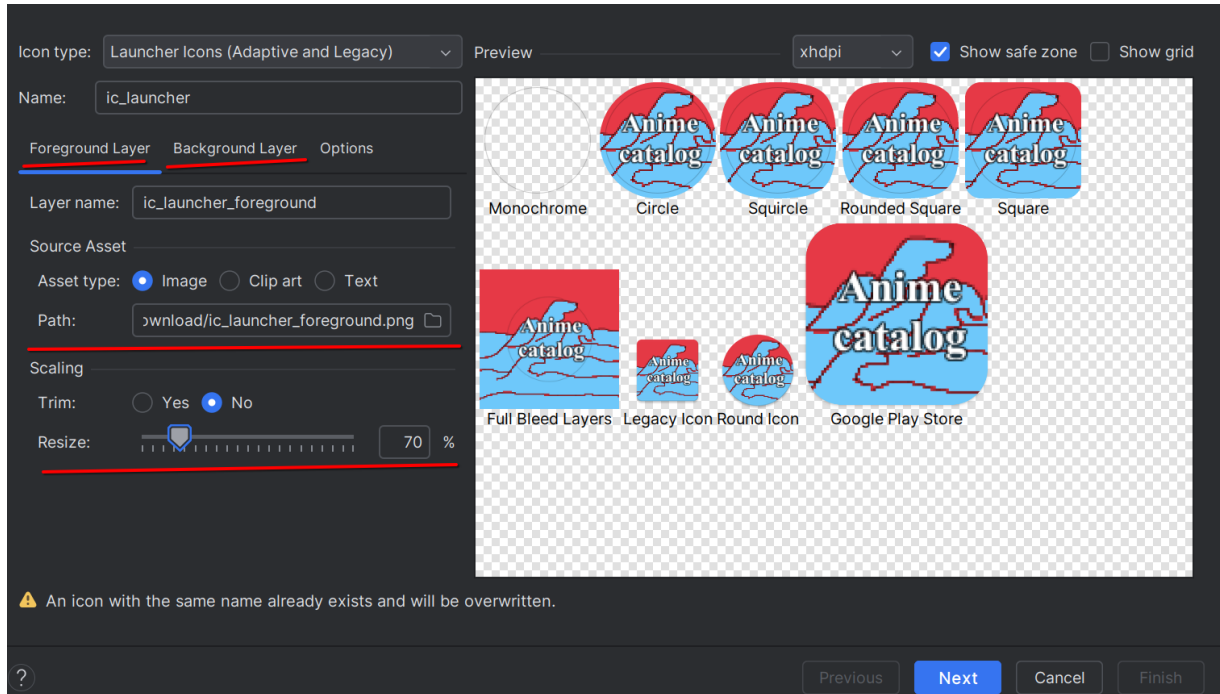
- Asset Type: Image
- Path: ic_launcher_background.png

Проверка превью

Посмотрите, как иконка выглядит:

- в круге
- в квадрате
- в скруглённом квадрате

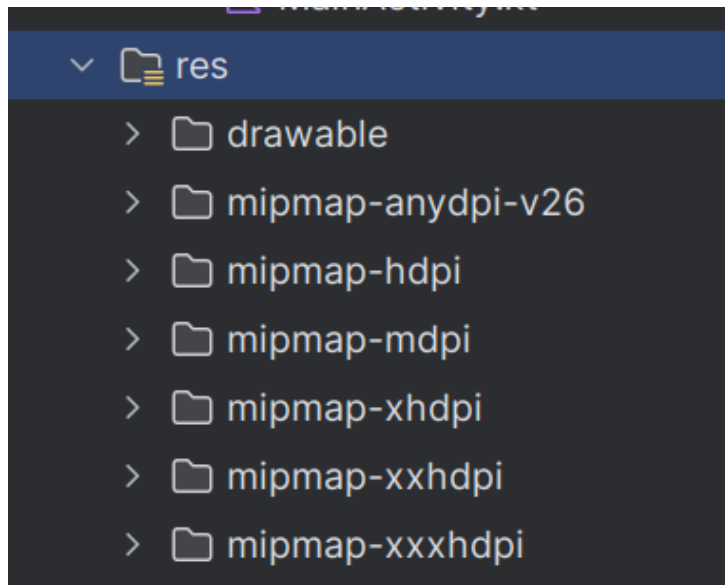
Если что-то обрезается — уменьшите Resize



Нажмите **Next** → **Finish**

8.6. Что создаст Android Studio

Автоматически появятся:



drawable/

- ic_launcher_background.xml
- ic_launcher_foreground.xml

mipmap-anydpi-v26/

- ic_launcher.xml
- ic_launcher_round.xml

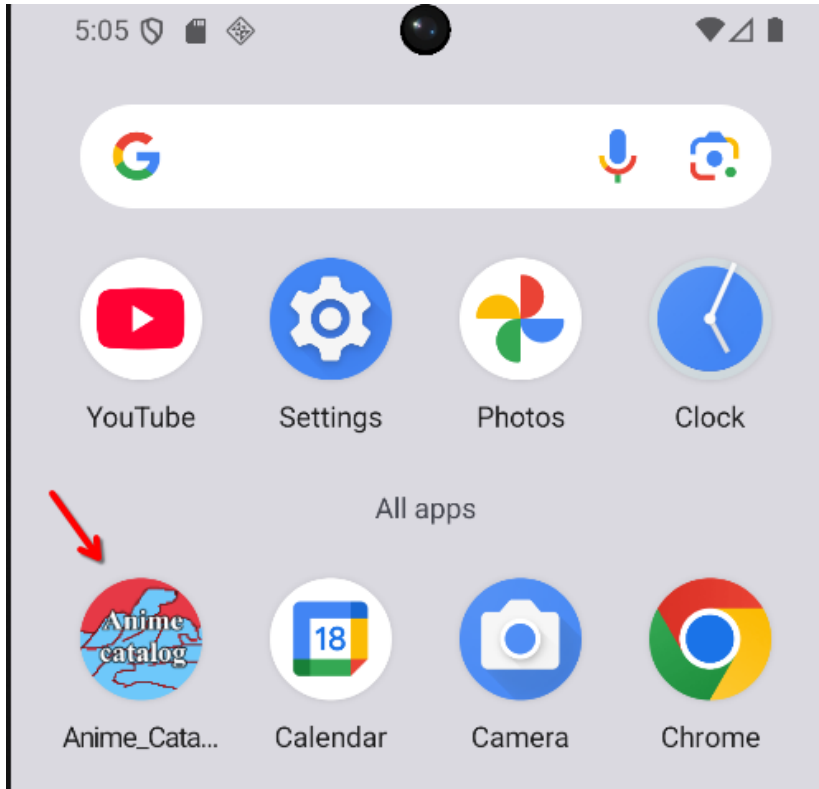
mipmap-*/

- иконки для всех плотностей

8.7. Проверка результата

1. Нажмите **Run**
2. Установите приложение
3. Нажмите **Home**
4. Найдите приложение **Anime Catalog**

Вы увидите **свою собственную аниме-иконку**



Шаг 9. Публикация проекта

Загрузите ваше приложение на репозиторий GitHub.

Шаг 10. Создание собственного каталога

Теперь, когда вы освоили создание приложения с прокручиваемым списком, пришло время применить полученные знания самостоятельно!

Требования к приложению

Ваше приложение должно включать:

- **10 элементов** в прокручиваемом списке
- **Изображения** для каждого элемента
- **Название и описание** для каждого элемента
- **Data class** для модели данных
- **Datasource** класс с данными
- **LazyColumn** для отображения списка
- **Card** компонент для каждого элемента
- **Собственную иконку** приложения
- **Локализованные строки** в strings.xml

Предлагаемые темы для выбора

Выберите одну из следующих тем или придумайте свою:

1. Game Catalog (Каталог игр)

- **Примеры:** Minecraft, GTA V, The Witcher 3, Cyberpunk 2077, Elden Ring
- **Данные:** Название игры, описание геймплея, изображение обложки
- **Особенность:** Можно добавить жанр игры

2. Movie Catalog (Каталог фильмов)

- **Примеры:** Inception, Interstellar, The Matrix, Avatar, The Shawshank Redemption
- **Данные:** Название фильма, краткое описание сюжета, постер
- **Особенность:** Можно добавить год выпуска

3. Book Catalog (Каталог книг)

- **Примеры:** Harry Potter, 1984, The Hobbit, Pride and Prejudice, Dune
- **Данные:** Название книги, описание, обложка
- **Особенность:** Можно добавить автора

4. Travel Destinations (Места для путешествий)

- **Примеры:** Париж, Токио, Нью-Йорк, Бали, Исландия
- **Данные:** Название места, описание достопримечательностей, фото

- **Особенность:** Можно добавить страну

5. Recipe Catalog (Каталог рецептов)

- **Примеры:** Пицца, Суши, Паста Карбонара, Тирамису, Борщ
- **Данные:** Название блюда, краткое описание, фото блюда
- **Особенность:** Можно добавить время приготовления

6. Music Albums (Музыкальные альбомы)

- **Примеры:** The Dark Side of the Moon, Abbey Road, Thriller, Nevermind
- **Данные:** Название альбома, описание, обложка
- **Особенность:** Можно добавить исполнителя

7. Cars Catalog (Каталог автомобилей)

- **Примеры:** Tesla Model S, Ferrari F8, Lamborghini Aventador, Porsche 911
- **Данные:** Модель автомобиля, описание характеристик, фото
- **Особенность:** Можно добавить производителя

8. Sports Catalog (Каталог видов спорта)

- **Примеры:** Футбол, Баскетбол, Теннис, Плавание, Бокс
- **Данные:** Название спорта, описание правил, изображение
- **Особенность:** Можно добавить количество игроков

9. Programming Languages (Языки программирования)

- **Примеры:** Kotlin, Python, Java, JavaScript, C++
- **Данные:** Название языка, описание применения, логотип
- **Особенность:** Можно добавить год создания

Пошаговая инструкция выполнения

Шаг 1: Выбор темы и подготовка

1. Выберите тему из списка или придумайте свою
2. Подберите **10 изображений** для вашей темы (размер: минимум 800x450px)
3. Придумайте **название** и **краткое описание** для каждого элемента

Шаг 2: Создание проекта

1. Создайте новый проект в Android Studio
2. **Name:** [Ваша_Тема]_Catalog (например, Game_Catalog)
3. **Package name:** com.example.[тема]catalog (например, com.example.gamecatalog)

4. **Minimum SDK:** API 25

Шаг 3: Подготовка ресурсов

1. Добавьте 10 изображений в res/drawable/
2. Переименуйте их: item1.jpg, item2.jpg, ..., item10.jpg
3. Создайте строки в strings.xml:
 - Названия элементов
 - Описания элементов

Шаг 4: Создание модели данных

1. Создайте пакет model
2. Создайте data class с **3 свойствами**:
 - ID названия (titleResourceId)
 - ID описания (descriptionResourceId)
 - ID изображения (imageResourceId)

Шаг 5: Создание источника данных

1. Создайте пакет data
2. Создайте класс DataSource
3. Добавьте функцию, возвращающую список из 10 элементов

Шаг 6: Создание интерфейса

1. Создайте функцию для отображения **одной карточки**
2. Создайте функцию с **LazyColumn** для списка
3. Создайте главную функцию приложения
4. Подключите в **MainActivity**

Шаг 7: Создание собственной иконки

1. Создайте уникальную иконку в стилистике вашей темы
2. Используйте Canva, Figma или Icon Kitchen
3. Добавьте через Image Asset Studio

Шаг 8: Тестирование

1. Запустите приложение
2. Проверьте прокрутку списка
3. Проверьте отображение всех элементов
4. Проверьте иконку приложения